

IoT Simulation & Monitoring Dashboard

Comprehensive Frontend Engineering & Deployment Guide

1. Technical Stack Overview

This frontend application is an enterprise-grade IoT device simulation and monitoring dashboard built using modern web development practices. It is engineered with high scalability, strict type safety, and optimized build workflows. The underlying core technologies include:

- **Core Library:** React 19.2.7 utilizing the concurrent rendering architecture.
- **Build System & Bundler:** Vite 8.1.0 configured with Rolldown for highly efficient, near-instantaneous compilation and hot module replacement.
- **Language:** TypeScript 6.0.3 enforcing comprehensive static typing and strict parameter checks.
- **Styling Engine:** Tailwind CSS v4.3.1 integrating natively into the Vite compilation layer through `@tailwindcss/vite`.
- **State Management:** Zustand 5.0.14 managing globally distributed slices for real-time telemetry datasets and UI components.
- **Data Visualization:** Recharts 3.9.0 handles modular, responsive client-side rendering of streaming time-series telemetry metrics.
- **Routing Architecture:** React Router v7.18.0 enabling declarative deep-linking and layout-driven route trees.
- **Animation Frameworks:** Framer Motion 12.42.0 and GSAP 3.15.0 managing micro-interactions, hardware-accelerated layouts, and complex physics-based canvas structures.

2. Local Environment Setup & Installation

To initialize the local workspace, verify that your development node possesses Node.js version 20.x or higher installed. Follow these strict sequence blocks to provision dependencies and start execution:

2.1 Initialization Commands

Execute the installation within the root folder containing the `package.json` archive. This maps exact dependency locks from the lockfile structure:

```
# Clean install exact locked dependencies
npm install
```

2.2 Available Task Routines

The build toolchain defines clear administrative commands detailed in the operational matrix below:

Routine Command	Underlying Execution Tool	Operational Description
<code>npm run dev</code>	Vite core compiler	Spins up the hyper-fast development local runtime container with hot reloading active.
<code>npm run build</code>	TypeScript Compiler (tsc) & Vite	Compiles project definitions using standard reference sub-trees and outputs to directory <code>/dist</code> .
<code>npm run lint</code>	ESLint Flat Configuration Engine	Runs static codebase analysis across code components to enforce structural safety standards.
<code>npm run preview</code>	Vite target runtime previewer	Instantiates a lightweight localized production proxy server serving build assets out of <code>/dist</code> .

3. Application Architecture & File Mapping

The system utilizes structured compiler configurations to maintain code cleanliness and ensure explicit pathway lookups through paths routing.

3.1 Path Aliasing Rules

The code uses path lookups via the custom asset pointer symbol (`@/*`). This ensures component relative path stability regardless of physical structural positioning. The binding maps as follows:

```
"@/*" : ["/src/*"]
```

This allows files inside the source directory to cleanly resolve resources like assets or stores via clean absolute root semantics instead of relative backtracking strings (`../../../../components`).

3.2 Configuration Boundaries

- **tsconfig.app.json** : Targets modern ECMAScript runtimes (`ES2022`), strictly blocking unmapped type emissions through `noEmit: true`, and ensuring code quality rules (`noUnusedLocals`, `noUnusedParameters`).
- **eslint.config.js** : Implements new flat config standard patterns, overriding TypeScript loose checks by maintaining explicit warn scopes on unused parameters matching the `^_` pattern to handle placeholder allocations cleanly.

Architecture Notice: State variables inside React hooks can bypass default triggers as structural checks on effect hook allocations have been set to relaxed configurations internally within the configuration file matrix.

4. Continuous Integration Pipeline (Azure DevOps)

Every operational commit targeting either the central stabilization trunk (`main`) or personal development pipelines (`test-(Vansh)`) triggers validation operations on the continuous integration layer managed inside the Azure DevOps pool matrix.

4.1 Pipeline Step Manifest

1. **Provision Execution Platform:** Pulls standard, clean `ubuntu-latest` execution environments.
2. **Runtime Injection:** Provisions Node.js version 20.x environment bindings natively.
3. **Dependency Resolution:** Executes an automated `npm install` script mapping package rules.
4. **Quality Check Rule:** Evaluates structural semantics by executing `npm run lint` . Any syntax rule violations crash compilation and halt deployment instantly.
5. **Distribution Compile Check:** Runs build verification routines through the Vite production build sequence engine to verify total compilation continuity.